Universidad Nacional Autónoma de México
Facultad de Ingeniería

Tarea #08.

Bases de Datos
(1644)

Nombre de estudiante:
PARRA FERNÁNDEZ HÉCTOR EMILIO

Número de cuenta: **321134567**

Número de brigada: **# 1**

Semestre: **2026-2**

Profesor:
Franco Arreola Fernando

Contenido

Tipos de Datos en PostgreSQL 1
Tipos de Datos Numéricos en PostgreSQL 1
1. Tipos Enteros (Integer Types) 1
2. Números de Precisión Arbitraria (Arbitrary Precision Numbers) 1
3. Tipos de Punto Flotante (Floating-Point Types) 1
4. Tipos Seriales (Serial Types) 2
Tipos de Datos de Caracteres en PostgreSQL 2
1. Tipos de Propósito General 2
2. Consideraciones Críticas de Rendimiento y Diseño 3
3. Arquitectura de Almacenamiento 3
4. Tipos Especiales (Uso Interno) 3
Tipos de Datos de Fecha y Hora en PostgreSQL 3
1. Tipos de Datos Principales 3
2. Entradas de Datos (Input) 4
3. Valores Especiales (Atajos) 4
4. Salida de Datos (Output) 5
Tipos de Datos Booleanos en PostgreSQL 5
1. Estados Lógicos (Tri-estado) 5
2. Entradas de Datos (Input) 5
1. Arquitectura y Propósito 6
2. Soporte y Generación de Versiones 6
3. Formatos de Entrada (Input) 7
4. Formato de Salida (Output) 7
Tipos de Datos JSON en PostgreSQL 7
1. Diferencias entre json y jsonb 7
2. Operaciones de Contención y Existencia (jsonb) 8
3. Indexación de Documentos (jsonb) 8
4. Consultas mediante jsonpath 8
Bibliography 10

Tipos de Datos en PostgreSQL

Tipos de Datos Numéricos en PostgreSQL

En PostgreSQL, los tipos de datos numéricos se dividen en cuatro categorías principales: enteros, números de precisión arbitraria, números de punto flotante y tipos seriales.

1. Tipos Enteros (Integer Types)

Almacenan números enteros sin componentes fraccionarios. Si se intenta almacenar un valor fuera del rango permitido, el sistema arrojará un error.

- smallint (2 bytes): Rango pequeño (-32,768 a +32,767). Se utiliza generalmente solo cuando el espacio en almacenamiento es crítico.
- integer (4 bytes): Rango estándar (-2,147,483,648 a +2,147,483,647). Es la elección más común y recomendada, ya que ofrece el mejor balance entre rango, tamaño de almacenamiento y rendimiento.
- bigint (8 bytes): Rango amplio. Se implementa cuando el rango del tipo integer resulta insuficiente para la cantidad de datos proyectada.

2. Números de Precisión Arbitraria (Arbitrary Precision Numbers)

Los tipos numeric y decimal son funcionalmente equivalentes y forman parte del estándar SQL.
Características: Almacenan números con una cantidad masiva de dígitos de forma exacta. Son el estándar requerido para almacenar montos monetarios o cualquier cantidad donde la exactitud matemática sea indispensable.

Rendimiento: Las operaciones aritméticas con estos tipos producen resultados exactos, pero su procesamiento es considerablemente más lento en comparación con los enteros o los tipos de punto flotante.

Sintaxis: Se declaran utilizando NUMERIC(precisión, escala). La *precisión* indica el número total de dígitos significativos en todo el número, mientras que la *escala* define la cantidad de dígitos permitidos a la derecha del punto decimal. Si se declara NUMERIC sin parámetros, permite almacenar números de cualquier longitud (dentro de los límites del sistema) sin forzar un redondeo.

Valores especiales: Soportan valores propios del estándar IEEE 754, tales como Infinity (infinito), -Infinity y NaN (Not-a-Number).

3. Tipos de Punto Flotante (Floating-Point Types)

Son tipos numéricos inexactos y de precisión variable, construidos sobre el estándar IEEE 754.

- real (4 bytes): Ofrece una precisión mínima de 6 dígitos decimales.
- double precision (8 bytes): Ofrece una precisión mínima de 15 dígitos decimales.

Características y riesgos: Al ser inexactos, algunos valores no pueden convertirse exactamente a su formato binario interno y se almacenan como aproximaciones. Esto significa que pueden existir discrepancias microscópicas y errores de redondeo. Son ideales y muy rápidos para cálculos matemáticos o científicos complejos, pero no deben utilizarse en contextos financieros.

Valores especiales: Al igual que la precisión arbitraria, soportan Infinity, -Infinity y NaN.

4. Tipos Seriales (Serial Types)

Los identificadores smallserial, serial y bigserial no son verdaderos tipos de datos en la arquitectura de PostgreSQL. Son una conveniencia notacional (azúcar sintáctico) para crear columnas identificadoras autoincrementables (similar al concepto de AUTO_INCREMENT en otros motores).

Tipos de Datos de Caracteres en PostgreSQL

En PostgreSQL, el manejo de cadenas de texto se realiza principalmente a través de tres tipos de datos de propósito general. Aunque dos de ellos están definidos por el estándar SQL, el comportamiento interno del gestor presenta particularidades importantes respecto a su rendimiento y almacenamiento.

1. Tipos de Propósito General

- character varying(n) / varchar(n): Almacena cadenas de longitud variable con un límite máximo definido por n (que representa cantidad de caracteres, no bytes).
    - Si se intenta insertar una cadena mayor al límite, el sistema arrojará un error. Sin embargo, existen dos excepciones: si los caracteres excedentes son únicamente espacios en blanco, la cadena se trunca silenciosamente al límite; y si se realiza un *cast* explícito (ej. ::varchar(n)), se fuerza el truncamiento sin generar error.
    - Si se utiliza varchar sin especificar una longitud n, la columna aceptará cadenas de cualquier tamaño.
- character(n) / char(n): Almacena cadenas de longitud fija.
    - Si la cadena insertada es menor que n, el sistema la rellena físicamente con espacios en blanco hasta alcanzar la longitud especificada.
    - Es crucial considerar que, aunque los espacios de relleno se almacenan, el gestor los ignora semánticamente al realizar comparaciones entre dos valores de tipo character.
    - Si no se especifica n, el gestor asume por defecto character(1).
- text: Es un tipo de dato de longitud variable y sin límite predefinido.
    - Aunque no forma parte del estándar SQL estricto, es el tipo nativo de manejo de cadenas en PostgreSQL (la mayoría de las funciones internas de texto operan sobre este tipo). Para la mayoría de los casos prácticos, varchar funciona internamente como un dominio sobre text.

2. Consideraciones Críticas de Rendimiento y Diseño

A diferencia de otros motores de bases de datos donde char(n) ofrece ventajas de rendimiento, en PostgreSQL no existe ninguna ventaja de optimización al usar tipos de longitud fija.

- De hecho, char(n) suele ser el tipo más lento y el que más espacio desperdicia debido a la necesidad matemática de calcular e insertar los espacios de relleno.
- *Recomendación oficial:* La documentación sugiere utilizar casi exclusivamente text o varchar sin límite, a menos que exista una restricción de negocio ineludible que obligue a limitar la longitud de la cadena.

3. Arquitectura de Almacenamiento

- El límite máximo real para almacenar una cadena en cualquier campo de texto de PostgreSQL es de aproximadamente 1 GB.
- Eficiencia dinámica: Para cadenas cortas (hasta 126 bytes), el sistema utiliza solo 1 byte de sobrecarga. Para cadenas largas, requiere 4 bytes de sobrecarga, pero el gestor las comprime automáticamente de forma transparente. Si los valores son extremadamente largos, se almacenan en tablas de fondo (fuera de la tabla principal) para no degradar los tiempos de escaneo secuencial.
- Sin importar el tipo de codificación de la base de datos, el único carácter que está estrictamente prohibido almacenar en estos tipos es el byte nulo (\0 o NUL).

4. Tipos Especiales (Uso Interno)

Existen dos tipos adicionales que no están diseñados para el modelado de datos general, sino para el funcionamiento de los catálogos del sistema de la base de datos:

- "char" (con comillas): Ocupa exactamente 1 byte físico y almacena un único carácter ASCII. Se utiliza internamente como una enumeración muy ligera.
- name: Ocupa 64 bytes y se utiliza exclusivamente a nivel interno del motor para almacenar los identificadores (nombres de tablas, columnas, índices, etc.).

Tipos de Datos de Fecha y Hora en PostgreSQL

PostgreSQL ofrece un soporte completo para los tipos de fecha y hora definidos por el estándar SQL, basándose en el calendario Gregoriano. La resolución máxima para las operaciones temporales es de 1 microsegundo.

1. Tipos de Datos Principales

- timestamp [without time zone] (8 bytes): Almacena simultáneamente la fecha y la hora sin considerar zonas horarias. Es el equivalente estándar cuando solo se escribe timestamp.
- timestamp with time zone (8 bytes): Almacena fecha y hora considerando la zona horaria. *(Nota: internamente siempre se convierte y almacena en UTC; la zona horaria se usa para la conversión al mostrar los datos).* Una abreviatura propia de PostgreSQL para este tipo es timestamptz.

- date (4 bytes): Almacena exclusivamente la fecha (año, mes, día), omitiendo cualquier componente de tiempo.
- time [without time zone] (8 bytes): Almacena exclusivamente la hora del día (sin fecha asociada).
- time with time zone (12 bytes): Almacena la hora del día con su respectiva zona horaria. La documentación advierte que su utilidad práctica es cuestionable y recomienda usar timestamp with time zone en su lugar.
- interval (16 bytes): Almacena periodos de tiempo o lapsos (ej. "1 año 2 meses 3 días").

Consideración de precisión: Los tipos time, timestamp e interval permiten especificar opcionalmente una precisión fraccionaria (p) para los segundos, con un rango permitido de 0 a 6.

2. Entradas de Datos (Input)

El motor de PostgreSQL es altamente flexible y acepta múltiples formatos de entrada (incluyendo ISO 8601, que es el recomendado, el estándar SQL y formatos tradicionales).

- **Literales de texto:** Cualquier valor de fecha u hora ingresado manualmente debe estar delimitado por comillas simples (ej. '1999-01-08').
- **Ambigüedad de fechas:** Para formatos donde el orden es ambiguo (ej. 1/8/1999), el sistema respeta la configuración del parámetro DateStyle, que puede ajustarse a MDY (mes-día-año), DMY (día-mes-año) o YMD (año-mes-día).
- **Manejo de Zonas Horarias:** Si se introduce un timestamp with time zone con una zona horaria explícita, el sistema lo convierte a UTC. Si no se especifica, asume la zona horaria definida en el parámetro TimeZone del sistema y luego lo convierte a UTC.

3. Valores Especiales (Atajos)

PostgreSQL soporta cadenas de texto reservadas que actúan como funciones o atajos al ingresar datos (se deben usar entre comillas simples):

- 'epoch': 1970-01-01 00:00:00+00 (el tiempo cero de Unix).
- 'infinity' / '-infinity': Representan un tiempo posterior o anterior a cualquier otra marca de tiempo posible.
- 'now': Marca el momento exacto en el que inició la transacción actual.
- 'today', 'tomorrow', 'yesterday': Representan la medianoche (00:00) del día correspondiente.

Precaución: No se recomienda usar valores como 'now' o 'tomorrow' en sentencias preparadas, vistas o funciones precompiladas, ya que el valor se evalúa y se "congela" en el momento de la compilación, volviéndose obsoleto. Para esos casos, se debe usar la función SQL equivalente (ej. CURRENT_TIMESTAMP o CURRENT_DATE).

4. Salida de Datos (Output)

El formato en el que PostgreSQL devuelve las fechas y horas está controlado por la configuración del servidor, permitiendo cuatro estilos principales:

- ISO: Es el formato por defecto y el requerido por el estándar SQL (ej. 1997-12-17 07:37:16-08).
- SQL (Ingres): Formato tradicional (ej. 12/17/1997 07:37:16.00 PST).
- POSTGRES: El estilo original del motor para Unix (ej. Wed Dec 17 07:37:16 1997 PST).
- German: Formato regional europeo (ej. 17.12.1997 07:37:16.00 PST).

Para los tipos interval, existe un comando propio (SET intervalstyle) que permite formatear la salida a estilos estandarizados, evitando la confusión entre componentes de tiempo con signos positivos y negativos.

Tipos de Datos Booleanos en PostgreSQL

PostgreSQL proporciona el tipo de dato boolean (que puede abreviarse como bool), diseñado para cumplir con el estándar SQL en el manejo de valores lógicos o de verdad.

1. Estados Lógicos (Tri-estado)

A diferencia de los lenguajes de programación tradicionales que operan con lógica binaria, el tipo booleano en PostgreSQL funciona con una "lógica de tres valores" (Three-Valued Logic). Una columna booleana puede adoptar uno de los siguientes estados:

- Verdadero (True)
- Falso (False)
- Desconocido (Unknown): Este tercer estado no tiene un valor literal propio, sino que se representa internamente mediante el valor NULL de SQL.

2. Entradas de Datos (Input)

El motor de PostgreSQL es notablemente flexible al recibir datos booleanos, aceptando diversas representaciones en formato de cadena de texto. El analizador (parser) ignora los espacios en blanco al inicio o final de la cadena, así como las diferencias entre mayúsculas y minúsculas.

- Valores aceptados como Verdadero: 'true', 't', 'yes', 'y', 'on', '1'.
- Valores aceptados como Falso: 'false', 'f', 'no', 'n', 'off', '0'.

Consideraciones de sintaxis y buenas prácticas: > 1. Las palabras clave TRUE y FALSE (sin comillas) son el método preferido y el exigido por el estándar SQL. Al usar estas palabras sin comillas, el analizador deduce automáticamente que son de tipo booleano. 2. Si se utilizan las representaciones alternativas (como 'yes' o '0'), estas deben tratarse como literales de cadena (con comillas simples) o castearse explícitamente (ej. 'yes'::boolean). 3. El valor NULL no tiene un tipo inherente, por lo que en ciertos contextos de ambigüedad, puede ser necesario castearlo como NULL::boolean.

3. Salida de Datos (Output)

A pesar de la flexibilidad permitida para la inserción de datos, PostgreSQL estandariza estrictamente la forma en la que los expone o devuelve en las consultas. La función de salida para este tipo de dato siempre emitirá:

- 't' para representar Verdadero.
- 'f' para representar Falso.
- Un espacio vacío o indicador de nulidad para el estado Desconocido (NULL).

4. Arquitectura de Almacenamiento

Independientemente del estado lógico que se esté almacenando en una fila (ya sea verdadero, falso o desconocido), el tipo de dato boolean es extremadamente ligero y ocupa exactamente 1 byte de espacio físico en disco.

Tipo de Dato UUID en PostgreSQL

El tipo de dato uuid está diseñado para almacenar Identificadores Únicos Universales (Universally Unique Identifiers), definidos por los estándares RFC 9562 e ISO/IEC 9834-8:2005. En algunos sistemas y entornos tecnológicos, este concepto también es conocido como GUID (Globally Unique Identifier).

1. Arquitectura y Propósito

- Estructura física: Un UUID es una cantidad de datos de exactamente 128 bits.
- Garantía de unicidad: Se generan mediante algoritmos diseñados para hacer matemáticamente improbable que dos personas o sistemas en el mundo generen el mismo identificador.
- Ventaja arquitectónica: A diferencia de los generadores de secuencias (como el tipo serial o las columnas IDENTITY), cuya unicidad se limita a una sola base de datos local, los UUID garantizan la unicidad a nivel global. Esto los convierte en la opción ideal y recomendada para sistemas distribuidos.

2. Soporte y Generación de Versiones

El estándar RFC 9562 define 8 versiones diferentes de UUID, cada una con distintos métodos de generación (basados en tiempo, nombres, o valores aleatorios).

- Almacenamiento agnóstico: El tipo de dato uuid de PostgreSQL puede almacenar cualquier UUID, independientemente de la versión o del sistema externo con el que fue creado.
- Generación nativa: PostgreSQL incluye soporte nativo integrado para generar identificadores por su cuenta utilizando los algoritmos UUIDv4 (basado enteramente en números aleatorios) y UUIDv7 (basado en el tiempo, lo cual mejora drásticamente el rendimiento al ser indexado en bases de datos).

3. Formatos de Entrada (Input)

El motor de PostgreSQL es tolerante y flexible al recibir cadenas de texto que representan un UUID, permitiendo ingresar un total de 32 dígitos hexadecimales bajo diversas variaciones sintácticas:

- El formato estándar: Secuencias separadas por guiones en grupos de 8-4-4-4-12 (ej. a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11).
- Inclusión de llaves: El formato estándar encerrado entre llaves (ej. {a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11}).
- Bloque continuo: Cadena sin ningún guion (ej. a0eebc999c0b4ef8bb6d6bb9bd380a11).
- Sensibilidad a mayúsculas: Se aceptan tanto letras mayúsculas como minúsculas.
- Guiones arbitrarios: Se permite la inserción de guiones después de cualquier grupo de cuatro dígitos.

4. Formato de Salida (Output)

A pesar de la extensa flexibilidad permitida para la inserción de los datos, el gestor de la base de datos estandariza de forma estricta cómo se exponen al usuario o a la aplicación. Al consultar un registro, la salida siempre se presentará en la forma estándar: todos los caracteres en minúscula y separados por guiones (8-4-4-4-12).

Tipos de Datos JSON en PostgreSQL

PostgreSQL ofrece soporte avanzado para el almacenamiento y procesamiento de datos en formato JSON (JavaScript Object Notation, RFC 7159). Aunque estos datos podrían almacenarse como texto simple (text), el uso de tipos JSON nativos garantiza la validación estricta del formato y permite el acceso a una amplia variedad de funciones y operadores de procesamiento.

El gestor proporciona dos tipos de datos principales para JSON: json y jsonb.

1. Diferencias entre json y jsonb

Ambos tipos aceptan prácticamente los mismos valores de entrada, pero su arquitectura interna y eficiencia varían drásticamente:

- json (Almacenamiento en texto plano):
    - Almacena una copia exacta de la cadena de texto de entrada.
    - Preserva el espacio en blanco (semánticamente insignificante) y el orden exacto de las claves del objeto.
    - Permite y almacena claves duplicadas (aunque en el procesamiento solo se toma la última).

  - *Desventaja:* Cada vez que se ejecuta una consulta o función de procesamiento, el motor debe volver a analizar (re-parsear) el texto completo.

- jsonb (Almacenamiento binario):
  - Almacena los datos en un formato binario descompuesto optimizado.
  - Elimina los espacios en blanco, no respeta el orden original de las claves y descarta automáticamente las claves duplicadas (conservando solo la última).
  - *Ventaja:* Aunque tiene una ligera sobrecarga al insertarse (por la conversión a binario), es significativamente más rápido para procesar y consultar porque no requiere re-parseo.
  - *Ventaja crítica:* Soporta indexación, lo cual es una diferencia abismal en rendimiento.
  - *Recomendación oficial:* Las aplicaciones deben preferir siempre jsonb sobre json, a menos que exista una necesidad heredada de mantener el orden exacto o el formato visual del texto.

2. Operaciones de Contención y Existencia (jsonb)

El tipo jsonb destaca por su capacidad nativa para realizar consultas complejas sobre la estructura de los documentos (funcionalidad no disponible en el tipo json clásico):

- Contención (@>): Permite verificar si un documento JSON contiene estructuralmente a otro. (Ej. {"a": 1, "b": 2} contiene a {"a": 1}). El orden y los elementos sobrantes no importan en la comprobación.
- Existencia (?, ?|, ?&): Permite comprobar rápidamente si una clave o un elemento de un arreglo existe en el nivel superior del documento JSON.

3. Indexación de Documentos (jsonb)

Para realizar búsquedas eficientes en bases de datos con millones de documentos JSON, PostgreSQL permite la creación de índices GIN (Generalized Inverted Index) sobre columnas jsonb. Existen dos clases de operadores principales para esto:

- jsonb_ops (Por defecto): Extrae y crea índices individuales para cada clave y valor del documento. Es muy flexible (soporta operadores de existencia y contención), pero los índices ocupan más espacio.
- jsonb_path_ops: Crea un "hash" que combina la clave y el valor. Esto genera un índice mucho más pequeño y búsquedas de contención (@>) notablemente más rápidas y precisas, pero pierde la capacidad de buscar solo por la existencia de una clave sin su valor.

4. Consultas mediante jsonpath

PostgreSQL incluye el tipo de dato jsonpath, diseñado para implementar un motor de consulta estructurada dentro de los documentos JSON.

- Utiliza una sintaxis híbrida basada en el estándar SQL/JSON y convenciones de JavaScript.
- Usa notación de puntos (.) para acceder a miembros de un objeto y corchetes ([]) para arreglos (los cuales empiezan en el índice 0, a diferencia de los arreglos nativos de SQL que inician en 1).
- Permite extraer datos profundamente anidados de forma elegante, soportando variables de contexto ($), comodines (.* para todos los miembros, .** para búsqueda recursiva en todos los niveles) y evaluación de filtros.

Bibliography

[1] PostrgresSQL, «Numeric Types,» [En línea]. Available: https://www.postgresql.org/docs/current/datatype-numeric.html.

[2] PostgresSQL, «Date/Time Types,» [En línea]. Available: https://www.postgresql.org/docs/current/datatype-character.html.

[3] Postgres, «Character Types,» [En línea]. Available: https://www.postgresql.org/docs/current/datatype-character.html.

[4] PostgresSQL, «Boolean Type,» [En línea]. Available: https://www.postgresql.org/docs/current/datatype-boolean.html.

[5] PostgresSQL, «UUID Type,» [En línea]. Available: https://www.postgresql.org/docs/current/datatype-uuid.html.

[6] Postgres, «JSON Types,» [En línea]. Available: https://www.postgresql.org/docs/current/datatype-json.html.